

Credit Name: Chapter 13
Assignment Name: StackList

How has your program changed from planning to coding to now? Please explain.

Initially, the plan was to implement a stack using a linked list that could store object data. The implementation followed the plan closely but encountered several challenges during development. These issues required modifications to ensure the program functioned correctly and handled edge cases effectively. Below are the specific problems faced and their solutions.

1. Forgetting to Initialize `top`

Problem:

The `top` pointer was not properly initialized, which caused a `NullPointerException` when attempting stack operations.

Fix:

I ensured that the `top` pointer was initialized to `null` in the constructor, representing an empty stack.

Code Example:

```
public StackList() {  
    top = null; // Initialize top to null  
}
```

2. Missing Link in Push Operation

Problem:

When adding a new item to the stack, the `next` pointer of the new node was not set to the current `top`, breaking the stack structure.

Fix:

I added logic to correctly link the new node to the current top of the stack before updating the `top` pointer.

```

public void push(Object item) {
    Node newNode = new Node(item); // Create a new node
    newNode.setNext(top);           // Link the new node to the current top
    top = newNode;                  // Update the top pointer
}

```

3. Handling Empty Stack in Pop Operation

Problem:

The `pop()` method did not handle empty stack scenarios properly, causing runtime errors.

Fix:

I added a condition to check if the stack is empty before attempting to remove an item, printing an error message if necessary.

Code Example:

```

public Object pop() {
    if (isEmpty()) { // Check if the stack is empty
        System.out.println("Stack is empty. Cannot pop.");
        return null;
    }
    Object item = top.getData(); // Get the data from the top node
    top = top.getNext();         // Update top to the next node
    return item;                 // Return the popped item
}

```

4. Inefficient Size Calculation

Problem:

The `size()` method traversed the entire stack every time it was called, making it inefficient.

Fix:

I introduced a `size` variable that increments or decrements during `push()` and `pop()` operations, enabling constant-time size retrieval.

Code Example:

```

private int size; // Add a size variable

public StackList() {
    top = null;
    size = 0; // Initialize size to 0
}

public void push(Object item) {
    Node newNode = new Node(item);
    newNode.setNext(top);
    top = newNode;
    size++; // Increment size when adding an item
}

public Object pop() {
    if (isEmpty()) {
        System.out.println("Stack is empty. Cannot pop.");
        return null;
    }
    Object item = top.getData();
    top = top.getNext();
    size--; // Decrement size when removing an item
    return item;
}

public int size() {
    return size; // Return the size variable directly
}

```

5. Error Handling in Peek Operation

Problem:

The `peek()` method did not handle empty stack scenarios, leading to runtime exceptions when accessing the top element.

Fix:

I added a condition to check if the stack is empty and return an error message instead of invalid data.

Code Example:

```

public Object peek() {
    if (isEmpty()) { // Check if the stack is empty
        System.out.println("Stack is empty. No top item.");
        return null;
    }
    return top.getData(); // Return the data at the top of the stack
}

```

Final Thoughts:

These adjustments improved the program's robustness and efficiency. By addressing issues such as linking nodes, handling empty stacks, and optimizing size calculation, the stack now functions correctly and efficiently. The program effectively demonstrates a stack implementation using a linked list while handling edge cases gracefully.